

SDLC Models — Complete Tier 1 Revision

Your syllabus/PDF covers these major models:

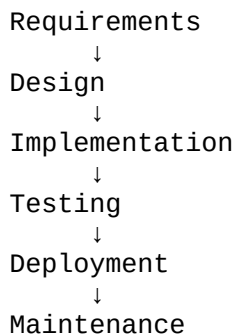
1. **Waterfall**
 2. **V-Model**
 3. **Iterative**
 4. **Incremental**
 5. **Spiral**
 6. **RAD**
 7. **Agile**
-

1. Waterfall Model

Definition

A sequential SDLC model in which development progresses through predefined phases, with each phase generally completed before the next begins.

Diagram



★ Key characteristic

Sequential + stable requirements

Advantages

- Simple and easy to understand
- Easy to manage
- Well-defined phases
- Good documentation
- Suitable when requirements are stable

Disadvantages

- Difficult to handle changes
- Customer feedback comes late
- Testing occurs relatively late
- Errors in requirements can become expensive to fix

When should I choose Waterfall?

Think:

"We know exactly what we need, and requirements won't change."

Case study

A government organization is developing payroll software. Requirements are clearly defined by regulations and are unlikely to change during development.

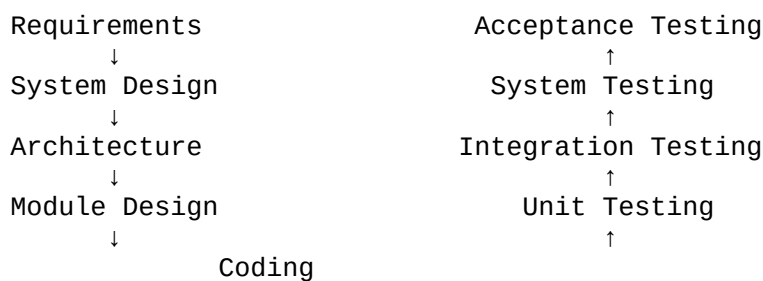
Answer → Waterfall

2. V-Model

Definition

A development model where each development phase has a corresponding testing/validation activity.

Diagram



★ Key characteristic

Verification + Validation / Testing

Advantages

- Testing planned early
- Clear relationship between development and testing
- Systematic

- Suitable for quality-critical systems

Disadvantages

- Difficult to accommodate changing requirements
- Inflexible
- Requires clear requirements from the beginning

When should I choose V-Model?

Think:

"Requirements are clear AND testing/validation is extremely important."

Case study

A medical device requires extensive verification and validation before it can be released.

Answer → V-Model

Memory:

Waterfall → Sequential

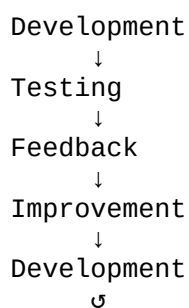
V-Model → Testing

3. Iterative Model

Definition

Software is developed through repeated cycles, with each cycle producing an improved version of the system.

Diagram



Key characteristic

Repeated improvement

Advantages

- Early working versions
- Feedback can be incorporated
- Supports changing requirements
- Continuous improvement

Disadvantages

- More management complexity
- Can increase cost/time
- Requires skilled team
- Scope may expand



When should I choose Iterative?

Think:

"We don't know everything initially, so we'll build something and keep improving it."

Case study

A company develops an application, releases an initial version, obtains user feedback, and repeatedly improves the existing functionality.

Answer → Iterative

Memory:

Iterative = Improve

4. Incremental Model

Definition

The system is developed and delivered in multiple increments, with each increment adding new functionality.

Diagram

```
Increment 1
  ↓
Basic Features
  ↓
Increment 2
  ↓
+ More Features
```

↓
Increment 3
↓
+ More Features
↓
Complete System

★ Key characteristic

Add functionality piece by piece

Example

An e-commerce application:

Increment 1 → Login
Increment 2 → Product Search
Increment 3 → Shopping Cart
Increment 4 → Payment
Increment 5 → Reviews



When should I choose Incremental?

Think:

"We can deliver the system feature-by-feature."

Case study

A company first releases login and registration, then adds product browsing, then payment, and finally order tracking.

Answer → Incremental

Memory:

Incremental = Add



Iterative vs Incremental

This is **extremely important**.

Iterative

Improve existing system
Focus = refinement
"Make it better"

Incremental

Add new functionality
Focus = addition
"Add another feature"

Remember:

ITERATIVE = IMPROVE
INCREMENTAL = ADD

And many modern approaches can actually be **both iterative and incremental**.

5. Spiral Model

Definition

A risk-driven iterative software development model in which development proceeds through repeated cycles while systematically identifying and addressing risks.

Diagram



★ Key characteristic

🔥 RISK ANALYSIS

Advantages

- Strong risk management
- Suitable for complex projects
- Supports changing requirements
- Continuous evaluation

Disadvantages

- Expensive
- Complex
- Requires risk-analysis expertise
- Not suitable for small/simple projects

🧠 When should I choose Spiral?

Think:

"This project has major technical, financial, safety, or other risks."

Case study

A company is developing a large defense system involving new technology and significant technical risks.

Answer → Spiral

Memory:

Spiral = Risk

6. RAD — Rapid Application Development

Definition

A software development model that emphasizes rapid development through prototyping, reusable components and strong user involvement.

Key characteristic

 **SPEED + PROTOTYPING**

Basic flow

Requirements
↓
User Design
↓
Construction
↓
Cutover

Advantages

- Fast development
- Early prototypes
- Frequent user feedback
- Reusable components

Disadvantages

- Requires skilled developers
- Requires strong user involvement
- Not suitable for every project

- Depends on suitable reusable components

When should I choose RAD?

Think:

"We need the application quickly and users can actively participate."

Case study

A business needs a working internal application within a few months. Users are available continuously to review prototypes and the system can be built using reusable components.

Answer → RAD

Memory:

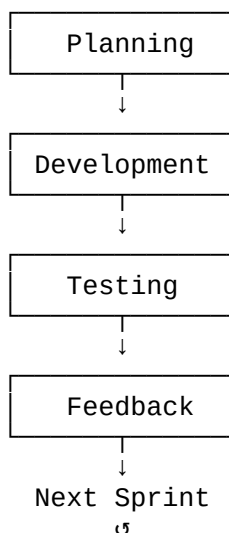
RAD = FAST

7. Agile Model

Definition

An iterative and incremental approach that emphasizes frequent delivery, customer collaboration, continuous feedback and responsiveness to changing requirements.

Diagram



★ Key characteristics

- Short iterations/sprints
- Frequent delivery
- Continuous customer feedback
- Welcomes changing requirements
- Collaboration

Advantages

- Highly flexible
- Early and frequent delivery
- Customer involvement
- Quickly responds to changing requirements

Disadvantages

- Requires strong collaboration
- Scope can change frequently
- Requires experienced team
- Cost/schedule can be less predictable

🧠 When should I choose Agile?

Think:

"Requirements will change frequently and the customer needs continuous involvement and frequent releases."

Case study

A startup is developing a mobile application. Customer requirements change frequently, users provide feedback after every release, and the team delivers new features every two weeks.

Answer → Agile

Memory:

Agile = CHANGE + FEEDBACK

🔥 NOW: How to identify the model in a case study

This is what you should memorize before the exam.

The Keyword Method

Case study clue	Model
Requirements fixed/stable	 Waterfall
Sequential phases	 Waterfall
Testing/validation is central	 V-Model
Repeatedly improve existing product	 Iterative
Add features in separate releases	 Incremental
High/critical risk	 Spiral
Need very fast development	 RAD
Frequent requirement changes	 Agile
Continuous customer feedback	 Agile
Short sprints	 Agile
Rapid prototyping	 RAD

The Ultimate Memory Trick

Memorize this sentence:

"Water falls, V tests, Iteration improves, Increment adds, Spiral manages risk, RAD is fast, Agile adapts."

Or simply:

WATERFALL → STABLE
V-MODEL → TESTING
ITERATIVE → IMPROVE
INCREMENTAL → ADD
SPIRAL → RISK
RAD → FAST
AGILE → CHANGE

If you remember **only this table**, you can identify most case studies.

Comparison Table — Learn This

Feature	Waterfall	V-Model	Iterative	Incremental	Spiral	RAD	Agile
Main focus	Sequential	Testing	Improvement	Feature addition	Risk	Speed	Adaptability
Change	 Low	 Low					 Very high
Feedback	Late	Limited	Regular	Regular	Regular	Frequent	Continuous
Risk analysis	Low	Low	Medium	Medium	 High	Medium	Continuous

Feature	Waterfall	V-Model	Iterative	Incremental	Spiral	RAD	Agile
Delivery	Usually later	Usually later	Early versions	Incremental	Iterative	Very fast	Frequent
Best clue	Stable requirements	Testing	Improve	Add	Risk	Speed	Change



Important Exam Trick

Sometimes **two models can appear correct**.

For example:

"Requirements are changing frequently and the team develops the system in iterations."

Both **Iterative** and **Agile** sound possible.

Look for the **stronger clue**:

If the question emphasizes:

"Repeatedly improve the product"

→ **Iterative**

If it emphasizes:

"Changing requirements + customer collaboration + short sprints + frequent releases"

→ **Agile**

Similarly:

"The system is delivered in several versions, each adding functionality."

→ **Incremental**

Not necessarily Agile.



What you should memorize NOW

You have limited time, so **don't memorize every paragraph**.

Write this on a piece of paper:

WATERFALL

→ Stable requirements

→ Sequential

V-MODEL

→ Testing / Validation

ITERATIVE

→ Improve repeatedly

INCREMENTAL

→ Add features

SPIRAL

→ Risk analysis

RAD

→ Rapid + Prototype

AGILE

→ Change + Feedback + Sprints

Then memorize the **7 diagrams at least roughly**.

Got it. Since you're **learning these topics for the first time**, I'll explain the concepts properly rather than giving you just revision bullets. But because of your time constraint, I'll keep each topic **exam-focused**—you'll understand it and also know exactly what to write.

We've finished the **SDLC models**. Let's move to the next Tier 1 topic:

Requirement Engineering

1. What is Requirement Engineering?

Imagine you're a software developer and a client tells you:

"I want a food delivery app."

Can you start coding?

No.

You need to understand exactly what the client means:

- Can users register?
- Can they search restaurants?
- Can they order food?
- Can they pay online?
- Can restaurants accept/reject orders?
- Should the system support 10,000 users?
- How quickly should it respond?
- What security is required?

The process of finding, understanding, documenting and managing these requirements is called **Requirement Engineering (RE)**.

Definition ★

Requirement Engineering is a systematic process of eliciting, analyzing, documenting, validating and managing the requirements of a software system.

2. Why do we need Requirement Engineering?

The biggest reason is:

A software system can be technically correct but still be useless if we build the wrong thing.

For example:

Customer wants:

"Online banking system where users can transfer money."

Developer misunderstands it as:

"Users only need to check their account balance."

Developer builds it perfectly.

But the customer doesn't get what they wanted.

So:

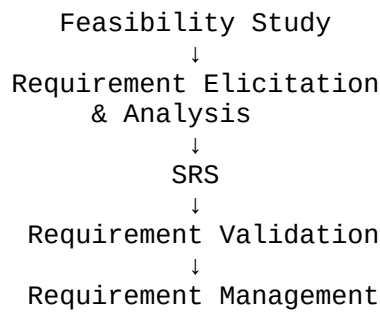
```
Wrong understanding
  ↓
Wrong requirements
  ↓
Wrong design
  ↓
Wrong software
  ↓
Rework + Cost + Time
```

Requirement Engineering tries to prevent this.

3. Requirement Engineering Process ★★

This is one of the most important things to understand.

The overall process can be represented as:



Now let's understand **why each step exists**.

4. Step 1 — Feasibility Study

Before building the software, we ask:

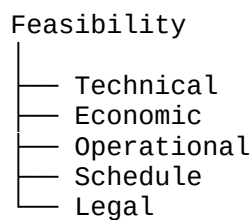
"Is it actually possible and worthwhile to build this system?"

For example, suppose a client wants a system that:

- costs ₹1 lakh
- must be completed in 1 month
- requires technology your organization doesn't have.

You need to determine whether the project is feasible.

Important feasibility types



Technical Feasibility

Can we build it using available technology, hardware and skills?

Economic Feasibility

Is it financially worthwhile?

Compare:

Expected benefits
vs
Development cost

Operational Feasibility

Will the organization/users be able to operate and use the system?

Schedule Feasibility

Can we finish it within the required time?

Legal Feasibility

Does the system comply with applicable laws and regulations?



Remember:

Technical = Can we build it?

Economic = Can we afford it?

Operational = Can we use it?

Schedule = Can we finish it on time?

Legal = Are we allowed to do it?

5. Step 2 — Requirement Elicitation and Analysis ★★

Now we need to **find out what the stakeholders actually want**.

This is called **requirement elicitation**.

What is elicitation?

Requirement elicitation is the process of gathering requirements from customers, users and other stakeholders.

Possible stakeholders:

Customer



Users



Managers



Domain Experts



Government / Regulators

How can we gather requirements?

Common techniques include:

- Interviews

- Questionnaires
 - Observation
 - Workshops
 - Brainstorming
 - Prototyping
 - Document analysis
-

6. Problems during Elicitation ★★

Your syllabus material identifies three important problems.

1. Scope Problem

The customer may provide too much unnecessary information or focus on implementation details instead of clearly defining the overall objective.

2. Understanding Problem

The customer and developer may understand the same requirement differently.

For example:

Customer:

"The system should be fast."

Developer:

"5 seconds is fast."

Customer:

"No, it should respond in less than 1 second."

The requirement was ambiguous.

3. Volatility Problem

Requirements can **change over time**.

Initial Requirement

↓

Customer changes requirement

↓

New requirement

↓

System needs modification

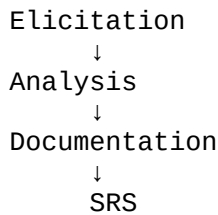
7. Step 3 — SRS

Once requirements are collected and analyzed, they need to be **documented**.

This is where SRS comes in.

SRS = Software Requirements Specification

It formally describes what the software is expected to do.



We'll study SRS separately in detail because it's an important exam topic.

8. Step 4 — Requirement Validation

Now we ask:

"Are these requirements actually correct?"

We check things such as:

Correctness

Does the requirement actually represent what the customer wants?

Completeness

Are any requirements missing?

Consistency

Do two requirements contradict each other?

Example:

R1 → Password must be at least 8 characters.

R2 → Password must be exactly 6 characters.

 Contradiction.

Unambiguity

Can the requirement have multiple interpretations?

Verifiability

Can we test whether the requirement has been satisfied?

9. Step 5 — Requirement Management

Requirements don't always stay fixed.

Suppose the customer initially says:

"Users should be able to pay using cards."

Later:

"Add UPI payments."

Later:

"Add wallet payments."

Someone needs to **track and manage these changes**.

That's Requirement Management.

Initial Requirements



Change Request



Analyze Impact



Approve / Reject



Update Requirements



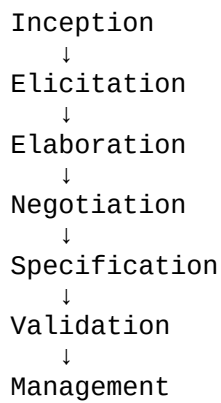
Track Changes

10. Seven Requirement Engineering Tasks

This is another thing your syllabus can ask.

Don't confuse this with the five-step process above.

The seven tasks are:



Let's understand each.

1. Inception

Understand:

- What problem are we solving?
- Who are the stakeholders?
- What is the overall objective?

Think:

"Understand the problem."

2. Elicitation

Gather requirements from stakeholders.

Think:

"Find out what they want."

3. Elaboration

Take the collected requirements and **refine/detail them**.

Think:

"Make the requirements clearer and more detailed."

Models and prototypes may be used.

4. Negotiation

Different stakeholders may have conflicting requirements.

Example:

Customer:

"Give me 30 features."

Developer:

"Budget allows only 15."

Manager:

"Deadline is 2 months."

Negotiation determines:

- Priorities
- Scope
- Cost
- Time
- Feasibility

Think:

"What can we realistically agree on?"

5. Specification

Document the agreed requirements.

Main output:

SRS

Think:

"Write it down."

6. Validation

Check whether requirements are:

- Correct
- Complete
- Consistent
- Unambiguous
- Verifiable

Think:

"Are these requirements correct?"

7. Management

Handle requirements throughout the project.

Think:

"Requirements change—track and control them."

11. Two Sequences You MUST Know

This is where students often get confused.

Overall Requirement Engineering Process

```
graph TD; A[Feasibility] --> B[Elicitation & Analysis]; B --> C[SRS]; C --> D[Validation]; D --> E[Management];
```

Seven RE Tasks

```
graph TD; A[Inception] --> B[Elicitation]; B --> C[Elaboration]; C --> D[Negotiation]; D --> E[Specification]; E --> F[Validation]; F --> G[Management];
```

How to remember the seven:

I Eat Every Nice Sandwich Very Much

I → Inception
E → Elicitation
E → Elaboration
N → Negotiation
S → Specification
V → Validation

12. Case Study — How to Identify the RE Activity

Situation 1

The development team talks with users to find out their requirements.

→ **Elicitation**

Situation 2

Two stakeholders disagree about which features should be included because of budget constraints.

→ **Negotiation**

Situation 3

The team creates a prototype to better understand and refine system requirements.

→ **Elaboration**

Situation 4

The team checks whether requirements are complete, consistent and testable.

→ **Validation**

Situation 5

The customer requests a new feature after requirements have already been approved.

→ **Requirement Management**

SRS — Software Requirements Specification

1. First understand: What exactly is an SRS?

Suppose a client tells a software company:

"I want an online shopping application."

This statement is **too vague** to start development.

The developer needs to know:

- Who can use the system?
- How do users register?
- How can they search products?
- How is an order placed?
- How is payment done?
- What happens if payment fails?
- How many users should the system support?
- How fast should it respond?
- What security is required?

All of these requirements need to be **clearly documented**.

That document is called the:

Software Requirements Specification (SRS).

Definition ★★ ★

SRS is a formal document that specifies the functional and non-functional requirements of a software system, along with relevant constraints, interfaces, and other requirements needed to understand what the system must provide.

Most important idea

SRS describes WHAT the system should do, not HOW developers should implement it.

2. Where does SRS come in the SDLC?

Remember what we just studied:

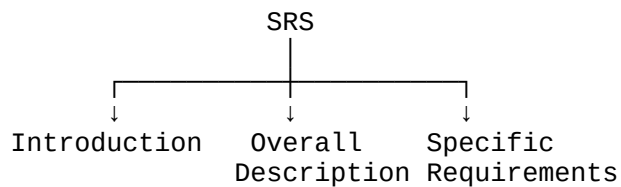
Requirements
↓
Analysis

↓
SRS
↓
Design
↓
Coding
↓
Testing

So SRS is produced **after requirements are elicited and analyzed** and is then used as a basis for design and development.

3. Structure of an SRS ★★

For your exam, understand this structure:



Let's understand each.

A. Introduction

This section gives the **basic context** of the software.

It generally contains:

1. Purpose

Why is this software being developed?

Example:

The purpose of this system is to provide an online platform for ordering food.

2. Scope

What is included in the system?

Example:

The system will allow users to browse restaurants, order food and make payments.

3. Definitions, Acronyms and Abbreviations

Defines technical terms used in the document.

Example:

OTP = One-Time Password

4. References

Documents or standards referred to while preparing the SRS.

5. Overview

Explains how the rest of the SRS is organized.

B. Overall Description

This gives a **high-level description of the complete system**.

Important parts include:

1. Product Perspective

How does this product relate to existing systems?

2. Product Functions

What major functions will the system provide?

Example:

User

- Register
- Login
- Search products
- Add to cart
- Make payment

3. User Characteristics

Who will use the system?

Example:

- Customer
- Administrator
- Seller

4. Constraints

Restrictions on the system.

Example:

- Must run on a particular platform
- Must comply with regulations
- Limited hardware resources

5. Assumptions and Dependencies

Things assumed to be true or external systems on which the software depends.

C. Specific Requirements

This is the **most detailed section**.

It describes the actual requirements that developers and testers need.

It can contain:

Specific Requirements

- Functional Requirements
- Non-functional Requirements
- External Interfaces
- Performance Requirements

This is where the actual behavior and constraints of the system are specified.

4. Functional Requirements

Now let's understand this properly.

Definition

Functional requirements describe the specific functions or services that the software must provide.

In simple words:

What should the system DO?

Example — Banking System

Functional requirements could be:

1. User shall be able to log in.
2. User shall be able to check account balance.
3. User shall be able to transfer money.
4. User shall be able to download account statements.
5. Admin shall be able to block an account.

These describe **actions/functions**.

Think:

FUNCTIONAL = WHAT THE SYSTEM DOES

5. Non-Functional Requirements

Definition

Non-functional requirements specify quality attributes, performance requirements, constraints, and other conditions under which the system must operate.

In simple words:

How well should the system perform?

Examples:

Performance

The system shall respond to a request within 2 seconds.

Security

User passwords shall be stored securely.

Availability

The system should be available 99.9% of the time.

Usability

A new user should be able to complete registration easily.

Scalability

The system should support increasing numbers of users.



Functional vs Non-Functional

This comparison is **almost guaranteed to be useful in the exam.**

Functional	Non-Functional
Describes what the system does	Describes how well / under what constraints it operates
Describes functions/services	Describes quality attributes and constraints
Usually feature-oriented	Usually quality/constraint-oriented
Example: User can transfer money	Example: Transfer must complete within 2 seconds
Example: User can search products	Example: Search results must appear within 1 second
Example: Admin can delete users	Example: System must provide secure authentication



One-line trick

Functional = WHAT

Non-functional = HOW WELL

6. Characteristics of a Good SRS

This is one of the **highest-priority things to memorize.**

A good SRS should be:

1. Correct

Requirements accurately represent the actual needs.

2. Unambiguous

Every requirement should have **only one interpretation.**



Bad:

The system should respond quickly.

What does "quickly" mean?



Better:

The system shall respond within 2 seconds.

3. Complete

All required functionality and relevant constraints should be included.

Nothing important should be missing.

4. Consistent

Requirements should not contradict each other.

Example:

R1 → Password must have at least 8 characters.

R2 → Password must contain exactly 6 characters.

These contradict each other.

 Not consistent.

5. Verifiable

We should be able to **test/check** whether the requirement has been satisfied.



The system shall be user-friendly.

Difficult to objectively test.



A new user shall be able to complete registration within 3 minutes.

Much easier to verify.

6. Modifiable

The document should be organized so that requirements can be changed without creating unnecessary confusion.

7. Traceable

Each requirement should be uniquely identifiable and traceable through development.

For example:

REQ-001



Design



Implementation



Test Case

8. Ranked

Requirements can be identified according to their:

- Importance
- Priority
- Stability

For example:

Login → High priority
Dark mode → Medium priority
Theme → Low priority

Architecture 4+1 View Model

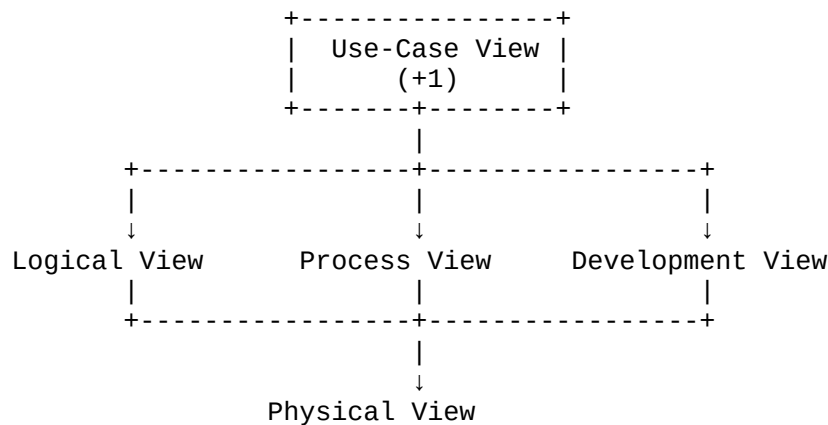
This is the **next topic in your System Architecture & Design section**. Learn the core idea first; then we'll build the exam answer around it.

1. What is the 4+1 View Model?

A software system is complex, and looking at it from only one perspective makes it difficult to understand.

The **4+1 View Model** describes a software architecture using **five different views**, where each view focuses on a different aspect of the system.

The five views are:



The five views:

1. **Logical View**
2. **Process View**
3. **Development View**

- 4. **Physical View**
 - 5. **Use-Case View (+1)**
-

2. Why is it called "4+1"?

There are **four architectural views**:

Logical
Process
Development
Physical

And there is one additional view:

Use-Case View

Therefore:

$$4 + 1 = 5 \text{ views}$$

The **Use-Case View** connects the other views by describing how users interact with the system.

3. Logical View

What does it describe?

The **Logical View** describes the system's major functional elements and how they relate to each other.

It focuses on:

What the system does from the user's functional perspective.

It commonly uses:

- Classes
- Objects
- Relationships
- Class diagrams

Example

For an online shopping system:

Customer
|
↓
Order

↓
Product



Remember:

Logical View → Functionality / Classes

4. Process View

The **Process View** focuses on the system's **runtime behavior**.

It describes:

- Processes
- Threads
- Communication
- Synchronization
- Concurrency

For example, an application might have:

```
User Request
  ↓
Web Process
  ↓
Payment Process
  ↓
Database Process
```

The important question is:

How do the different processes execute and communicate at runtime?



Remember:

Process View → Runtime / Processes

5. Development View

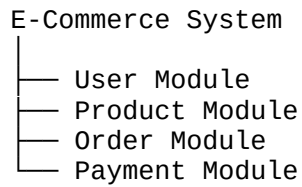
The **Development View** describes how the software is organized from the **developer's perspective**.

It focuses on:

- Modules
- Components

- Packages
- Subsystems
- Organization of software

For example:



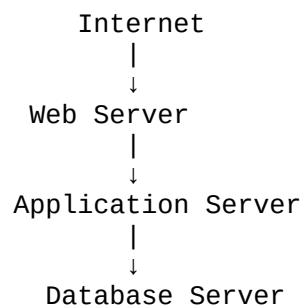
Remember:

Development View → Code organization

6. Physical View

The **Physical View** describes how software components are **deployed onto physical hardware/infrastructure**.

For example:



It is concerned with:

- Deployment
- Hardware
- Servers
- Network connections
- Distribution of components



Remember:

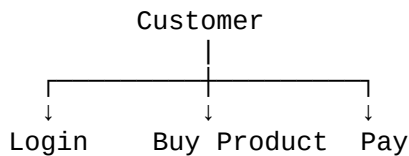
Physical View → Deployment / Hardware

7. Use-Case View (+1) ★★

This is the **+1 view**.

It describes the system from the perspective of **users and their interactions with the system**.

For example:



It uses **use cases** to represent important interactions.

The Use-Case View is especially useful for ensuring that the architecture satisfies the required functionality.



Remember:

Use-Case View → User interactions



8. The Most Important Memory Trick

You need to remember what each view focuses on:

LOGICAL	→ WHAT functions/classes?
PROCESS	→ HOW does it run?
DEVELOPMENT	→ HOW is the software organized?
PHYSICAL	→ WHERE is it deployed?
USE-CASE	→ HOW does the user interact?

Or memorize:

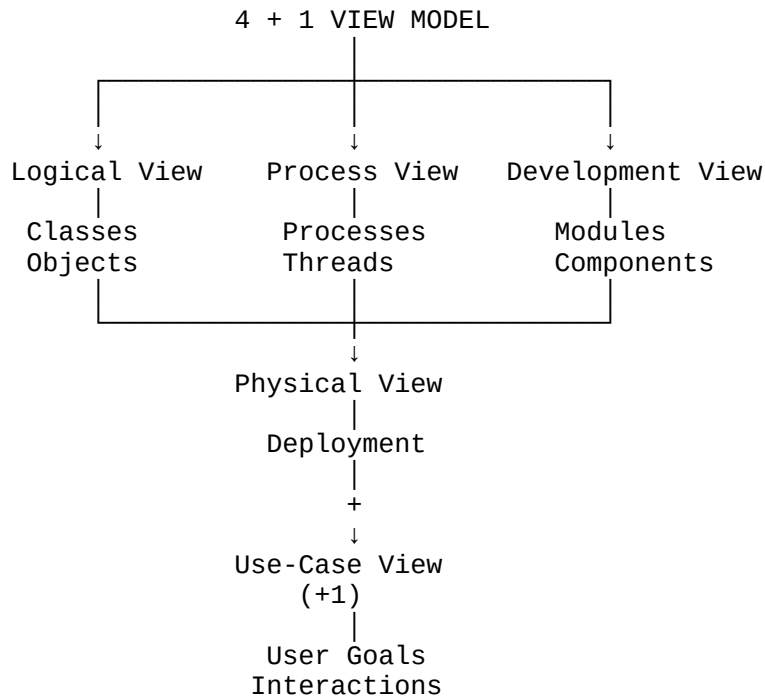
Logical = Function
Process = Runtime
Development = Code
Physical = Deployment
Use Case = User

9. Complete Diagram for Exam ★★

If asked:

"Explain the 4+1 architectural view model."

Draw this:



★ 10. Comparison Table

View	Main focus	Think
Logical	Functionality, classes, objects	WHAT
Process	Runtime processes, communication	RUN
Development	Modules/components/packages	CODE
Physical	Hardware and deployment	WHERE
Use-Case (+1)	User interactions	USER

★ 11. How to Identify a View in a Case Study

Case 1

The architect wants to represent Customer, Order and Product classes and their relationships.

Answer → Logical View

Case 2

The architect wants to show concurrent processes and how they communicate during execution.

Answer → Process View

Case 3

The development team wants to organize the system into packages and modules.

Answer → Development View

Case 4

The architect wants to show which components are deployed on the web server and database server.

Answer → Physical View

Case 5

The architect wants to represent how customers interact with the system to place an order.

Answer → Use-Case View

